

推荐统一建模指南

从特征交互、序列建模到 OneTrans / HyFormer

OpenAI 整理

2026-04-21

目录

适用对象	4
1 先用一句人话概括这件事	4
2 什么叫特征交互模型	5
2.1 先看一个最直观的例子	5
2.2 为什么光看单个字段不够	6
2.3 这一条路线的代表模型	6
2.4 这条路线的优点	14
2.5 这条路线的不足	14
3 什么叫序列模型	14
3.1 还是用同一个电商例子	14
3.2 为什么序列这么重要	15
3.3 最简单的序列想法：把历史求平均	15
3.4 DIN：让候选商品去“问”历史	16
3.5 DIEN：不只看相关性，还看兴趣怎么演化	18
3.6 BST / SASRec / BERT4Rec：用 Transformer 看序列	20
3.7 这条路线的优点	24
3.8 这条路线的不足	24
4 什么叫“两条路线单独演进”	24
4.1 一个典型旧架构长这样	24
4.2 为什么会这样	25
4.3 “单独演进”具体体现在哪	25
4.4 这种分离为什么会成为瓶颈	27
5 统一建模到底在统一什么	28

5.1	先搭一个统一视角：推荐排序的输入到底长什么样	28
5.2	统一可以发生在四层	28
5.3	为什么统一建模更自然	30
5.4	统一建模不等于“把所有东西粗暴拼接”	30
6	OneTrans：把序列 token 和非序列 token 放进同一条 Transformer 流	30
6.1	OneTrans 最核心的想法	30
6.2	第一步：统一 token 化	30
6.3	第二步：放进同一个 causal Transformer block	33
6.4	第三步：Mixed parameterization	33
6.5	第四步：causal mask 到底在统一什么	34
6.6	第五步：Pyramid Stack	34
6.7	第六步：Cross-request / Cross-candidate KV caching	35
6.8	OneTrans 到底统一在哪	35
6.9	OneTrans 的优势与代价	35
7	HyFormer：不是把所有 token 硬塞一条流，而是用 Global Query 做统一接口	36
7.1	HyFormer 的出发点	36
7.2	HyFormer 的总体图景	36
7.3	第一步：Query Generation	38
7.4	第二步：Sequence Representation Encoding	38
7.5	第三步：Query Decoding	39
7.6	第四步：Query Boosting	39
7.7	第五步：逐层交替，形成 alternating optimization 风格的统一骨干	40
7.8	HyFormer 为什么不把多条序列都拼成一条大流	40
7.9	HyFormer 的工程优化	40
7.10	HyFormer 到底统一在哪	41
7.11	HyFormer 的优势与代价	41
8	OneTrans 和 HyFormer 的核心差异，逐点掰开说	41
8.1	一张对照表先看清楚	41
8.2	信息流到底怎么走	42
8.3	为什么 HyFormer 不满意 OneTrans 这类“全流自注意力统一”	42
8.4	为什么 OneTrans 不想走 HyFormer 那么“模块化”的路	43
9	用一个具体样本来理解两者的差别	43
9.1	OneTrans 会怎么做	43
9.2	HyFormer 会怎么做	43
9.3	这两个直觉最本质的不同	44

10	如果把它们映射回你熟悉的 CTR / 排序框架	44
10.1	它们都在反对什么	44
10.2	OneTrans 的映射方式	44
10.3	HyFormer 的映射方式	45
10.4	对“统一建模”这个命题，它们分别给出的答案	45
11	回到工业落地与 TAAC 这类赛题，该怎么理解	45
11.1	第一，为什么这类任务天然适合统一建模	45
11.2	第二，为什么单模型、单链路很重要	45
11.3	第三，不是 AUC 最大就一定能上线	45
11.4	第四，统一建模不是让你一上来就堆超大模型	46
12	训练策略应该怎么想	46
12.1	先明确你优化的到底是什么	46
12.2	数据切分一定要按时间	46
12.3	序列构造要避免“偷看答案”	46
12.4	长序列不一定越长越好	46
12.5	类别不平衡、多目标训练、时延约束都要谨慎	47
13	评估策略应该怎么做	47
13.1	不要只盯着 leaderboard	47
13.2	建议同时看这些指标	47
13.3	做分桶评估	47
13.4	做消融实验	47
14	一个务实的实验起步路径	48
14.1	Baseline 1: 强表格基线	48
14.2	Baseline 2: 加一个轻量序列模块	48
14.3	Baseline 3: 往统一建模推进	48
14.4	为什么不建议新手一上来就上超大统一模型	48
14.5	如果你准备开始做实验，最建议先做的 5 件事	48
15	给初学者的一个最终心智模型	48
16	参考资料与建议阅读	49
16.1	经典路线	49
16.2	统一建模方向	49
16.3	建议怎么读	49
16.4	最后一段话	49

适用对象

- 第一次系统接触推荐排序、CTR / CVR 预估、特征交互、序列建模、统一建模的读者。
- 已经理解 DLRM、DIN、SASRec 这类模型名字，但还没有真正串成一个完整心智模型的读者。
- 想把 OneTrans、HyFormer 映射回工业排序系统，而不是只停留在“看过论文名”的读者。

1 先用一句人话概括这件事

推荐系统本质上是在回答一个问题：

给定一个用户、一个候选内容、一个发生场景，以及用户过去的历史行为，当前这次推荐成功的概率有多大？

把它写成数学形式，就是：

$$\mathbf{x} = (u, i, c, h)$$

其中：

- u 表示用户特征；
- i 表示候选物品或广告特征；
- c 表示上下文特征；
- h 表示用户历史行为序列。

模型输出一个分数：

$$s = f_{\theta}(u, i, c, h)$$

再把分数映射成概率：

$$p = \sigma(s) = \frac{1}{1 + e^{-s}}$$

如果是最常见的点击率或转化率建模，训练时最基本的损失就是二元交叉熵：

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{n=1}^N \left[y_n \log p_n + (1 - y_n) \log(1 - p_n) \right].$$

所以你可以把推荐建模先理解成三件事：

- 输入是一大堆用户、物品、上下文、历史行为信息；
- 输出是一个分数；

- 排序时按分数从高到低推荐。

看起来很简单，但真正难的是：这些输入不是同一种东西。

- 有些是静态特征，比如性别、年龄段、城市、设备；
- 有些是候选 item 的属性，比如类目、品牌、价格带；
- 有些是上下文，比如时间、流量位置、频道；
- 有些是长序列，比如用户最近点击过什么、买过什么、停留过什么。

这就自然分裂出了两条技术路线：

- 一条更像“表格建模”，重点看字段之间怎么交互；
- 一条更像“时序建模”，重点看历史行为是怎么演化的。

2 什么叫特征交互模型

2.1 先看一个最直观的例子

假设我们在做电商推荐。某次样本是：

- 用户：25 到 34 岁，女性，上海，iPhone；
- 候选商品：跑鞋，品牌 Nike，价格带 300 到 500；
- 上下文：晚上 10 点，推荐流第 3 位；
- 标签：最后下单了。

如果我们只看这一行样本，它其实很像一张表：

字段类型	例子
用户特征	年龄段、性别、城市、设备
商品特征	商品 ID、类目、品牌、价格带
上下文特征	时间段、流量位置、页面类型
标签	是否点击、是否转化

特征交互路线会把这类问题理解为：

哪些字段和哪些字段的组合，最能解释用户会不会点击或转化？

比如模型可能学到这些规律：

- 女性 × 运动品类更容易转化；
- 晚上 × 手机端 × 鞋类的点击更高；
- 上海 × 品牌运动品 × 中高价格带的购买意愿更强。

这里最关键的词就是“交互”。它不是只看单个字段，而是看字段之间的组合关系。

2.2 为什么光看单个字段不够

如果模型只看单个字段，可能只能学到：

- 女性整体更爱买；
- 晚上整体更容易下单；
- 运动商品整体更热门。

但真实世界里更常见的是组合效应：

- 不是所有女性都喜欢所有商品；
- 不是所有晚上都更容易下单；
- 不是所有运动商品都适合所有城市和设备场景。

真正起作用的是：

用户特征 × 商品特征 × 上下文特征.

这就是特征交互模型想重点解决的事。

2.3 这一条路线的代表模型

2.3.1 逻辑回归与人工交叉

最早大家会做手工交叉特征，比如：

- gender=female AND cate=sports
- city=shanghai AND device=iphone
- hour=night AND slot=3 AND brand=nike

然后用逻辑回归：

$$s = w_0 + \sum_{j=1}^m w_j x_j$$

再通过 sigmoid 变成概率：

$$p = \sigma(s) = \frac{1}{1 + e^{-s}}.$$

这套做法背后的逻辑非常朴素：

- 把一条样本拆成很多个特征；
- 每个特征各自贡献一点分数；
- 把所有贡献加起来，得到总分。

如果把一条样本写成：

$$\mathbf{x} = [x_1, x_2, \dots, x_m],$$

那么逻辑回归其实是在做一个“加法打分表”。但问题马上就来了：真实世界里很多模式不是“简单相加”，而是“组合后额外发生变化”。

更直白一点，你可以把它想成：

- 第 1 个特征乘自己的权重；
- 第 2 个特征乘自己的权重；
- 以此类推；
- 最后全部加起来。

它特别像一个“打分表”：

- 女性加 0.2 分；
- 运动类商品加 0.3 分；
- 晚上加 0.1 分。

于是模型能表达：

$$\text{总分} = \text{女性贡献} + \text{运动类贡献} + \text{晚上贡献}.$$

例如：

- 女性本身不一定特别爱点广告；
- 运动类本身也不一定特别高点击；
- 但“女性 × 运动类”这个组合可能明显更强。

所以大家开始手工构造交叉特征：

$$x_{\text{gender} \times \text{cate}} = x_{\text{gender}} \cdot x_{\text{cate}}$$

此时模型就能写成：

$$s = w_0 + w_1 x_{\text{gender}} + w_2 x_{\text{cate}} + w_3 x_{\text{gender} \times \text{cate}}.$$

这其实就是特征交互路线最原始的思想：单个特征不够，要看特征组合。

但问题也很明显：

- 手工组合太多；
- 工程维护困难；
- 看不到没人工构造出来的组合。

尤其在真实工业场景里，字段可能有几十上百个。如果你想手工构造：

- 两两交叉；
- 三三交叉；
- 不同字段的桶化交叉；

参数和工程复杂度都会迅速爆炸。

2.3.2 FM

FM 的关键思想是：不用手工枚举所有组合，而是给每个特征学一个向量，再用向量内积表示二阶交互强度。

$$\hat{y} = w_0 + \sum_{j=1}^m w_j x_j + \sum_{j=1}^m \sum_{k=j+1}^m \langle \mathbf{v}_j, \mathbf{v}_k \rangle x_j x_k.$$

这里：

- w_j 是一阶权重；
- $\mathbf{v}_j$ 是第 j 个特征的隐向量；
- $\langle \mathbf{v}_j, \mathbf{v}_k \rangle$ 表示两个特征的交互强度。

把这个公式拆开看，它其实分成两部分。

第一部分：

$$w_0 + \sum_{j=1}^m w_j x_j$$

这是逻辑回归的一阶部分，也就是“每个特征单独贡献多少分”。

第二部分：

$$\sum_{j=1}^m \sum_{k=j+1}^m \langle \mathbf{v}_j, \mathbf{v}_k \rangle x_j x_k$$

这是二阶交互部分，也就是“任意两个特征一起出现时，会不会产生额外作用”。

为什么用内积来表示交互强度？因为模型想做的不是为每一对特征单独存一个参数表，而是给每个特征学一个低维表示，让相似特征共享统计强度。

你可以把它理解成：

- 上海这个特征有一个 embedding；
- Nike 这个特征也有一个 embedding；
- 它们的内积越大，表示“上海用户和 Nike 的组合更匹配”。

举个极小例子。如果：

$$\mathbf{v}_{\text{shanghai}} = [1, 2], \quad \mathbf{v}_{\text{nike}} = [3, 4],$$

那么交互强度就是：

$$\langle \mathbf{v}_{\text{shanghai}}, \mathbf{v}_{\text{nike}} \rangle = 1 \times 3 + 2 \times 4 = 11.$$

如果当前样本里：

$$x_{\text{shanghai}} = 1, \quad x_{\text{nike}} = 1,$$

那这对交互就会给分数贡献：

$$11 \times 1 \times 1 = 11.$$

如果其中一个特征没出现，比如：

$$x_{\text{shanghai}} = 1, \quad x_{\text{nike}} = 0,$$

那么这项贡献就是：

$$11 \times 1 \times 0 = 0.$$

这就非常符合我们对交互的直觉：两个特征都出现，交互才有意义。

FM 在推荐里特别重要，是因为很多输入是高维稀疏特征：

- 用户 ID；
- 商品 ID；
- 城市；
- 品牌；
- 类目；

- 设备。

如果全部 one-hot 之后，大部分维度都为 0。FM 的优点恰恰在于：

- 不需要手工指定谁和谁交叉；
- 不需要为每一对组合单独存一个参数；
- 只需要为每个特征学一个 embedding。

2.3.3 Wide & Deep

Google 的 Wide & Deep 把“记忆”和“泛化”拆开：

- Wide 部分记住高频共现模式；
- Deep 部分通过 embedding 和 MLP 泛化到没见过的组合。

一个简化版公式可以写成：

$$s = \mathbf{w}_{\text{wide}}^\top \mathbf{x}_{\text{wide}} + \mathbf{w}_{\text{deep}}^\top \mathbf{a}^{(L)} + b.$$

其中：

- $\mathbf{x}_{\text{wide}}$ 是 wide 分支输入，通常包含原始特征或手工交叉特征；
- $\mathbf{a}^{(L)}$ 是 deep 分支最后一层隐藏表示；
- wide 分支负责记忆高频模式；
- deep 分支负责做非线性泛化。

deep 分支内部通常是：

$$\mathbf{a}^{(1)} = \phi(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}), \quad \mathbf{a}^{(2)} = \phi(\mathbf{W}^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)}),$$

一直堆到最后一层。

你可以把 Wide & Deep 理解成对工业现实的一个折中：

- 有些模式经常出现，而且很稳定，适合直接记住；
- 有些模式不常出现，但和已知模式相似，适合通过 embedding 和 MLP 去泛化。

为什么这很重要？因为真实推荐里既有：

- 高频模式，例如苹果手机用户在晚间更爱点数码配件；

也有：

- 长尾模式，例如一个新商品虽然没什么点击历史，但和某些热门商品很像。

Wide & Deep 的价值就在于：既不放弃强记忆能力，也不放弃深度模型的泛化能力。

2.3.4 DLRM

Meta 的 DLRM 可以看成工业级大规模稀疏推荐模型的经典范式：

1. 每个离散字段先做 embedding；
2. 稠密特征走一个 bottom MLP；
3. embedding 向量之间做显式交互；
4. 再接 top MLP 输出分数。

如果把它写成更接近公式的流程，通常是：

第一步，离散特征查 embedding：

$$\mathbf{e}_j = \text{Emb}_j(x_j)$$

第二步，稠密特征过 bottom MLP：

$$\mathbf{z}_{\text{dense}} = \text{MLP}_{\text{bottom}}(\mathbf{x}_{\text{dense}})$$

第三步，把稠密表示和所有离散 embedding 放在一起：

$$\mathbf{Z} = [\mathbf{z}_{\text{dense}}, \mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_m]$$

第四步，计算显式交互，最常见的方式是两两内积：

$$g_{ij} = \langle \mathbf{Z}_i, \mathbf{Z}_j \rangle$$

第五步，把这些交互结果与原始表示一起送入 top MLP：

$$s = \text{MLP}_{\text{top}}([\mathbf{z}_{\text{dense}}, \{g_{ij}\}]).$$

DLRM 和 Wide & Deep 的区别在于：

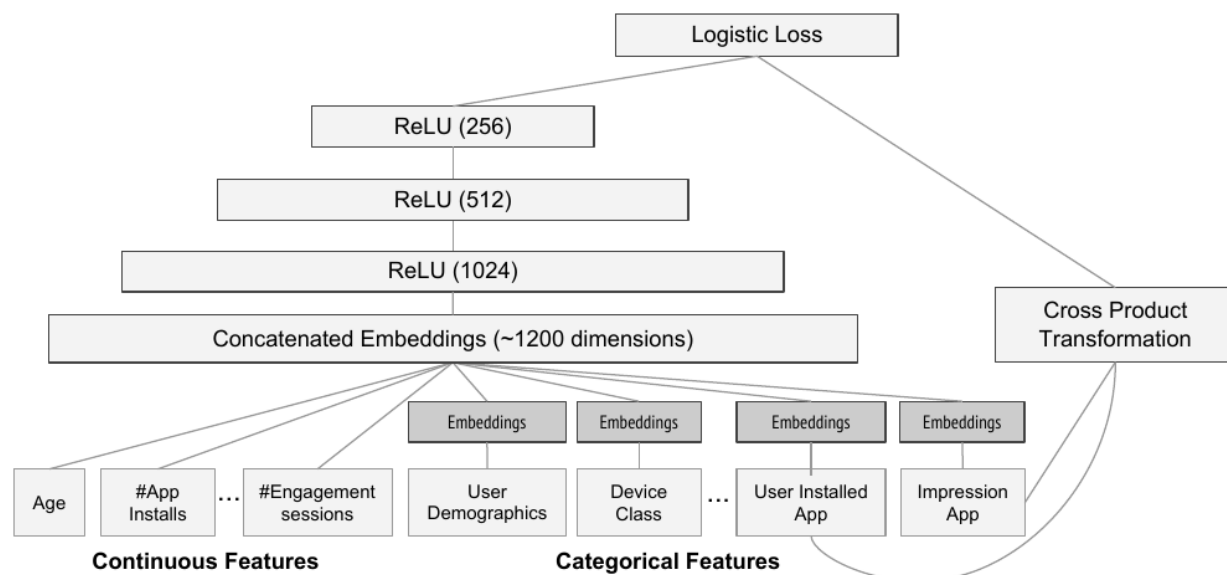
- Wide & Deep 更像 “wide 分支 + deep 分支” 的两路结构；
- DLRM 更像 “embedding + 显式交互 + MLP” 的工业流水线。

为什么 DLRM 会成为很多工业系统的基线？因为它非常贴近真实广告和推荐系统的输入形态：

- 大量离散字段；
- 少量稠密字段；

Wide & Deep: 显式交叉与深层泛化如何并联

只用文字描述时，Wide & Deep 很容易被理解成“一个 wide 分支加一个 deep 分支”。论文原图更能帮助我们看清，这两个分支并不是简单叠加，而是在同一套推荐输入上分担了两类截然不同的任务。



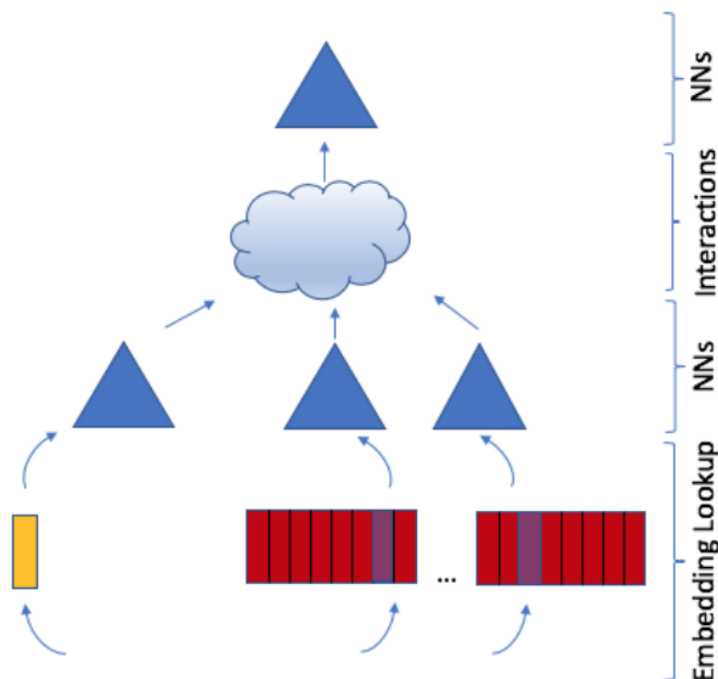
Wide & Deep 模型结构。引自《Wide & Deep Learning for Recommender Systems》Figure 4。

图里最直观的地方，是 wide 侧把交叉后的特征直接连到输出，而 deep 侧则把连续特征和类别特征先表示成稠密向量，再交给多层感知机去学习高阶组合。正文里说的“记忆”和“泛化”，在这里对应的正是这两条信息路径的不同分工。

因此，Wide & Deep 的关键不在于层数有多深，而在于它保留了一条显式交叉的捷径。模型既可以把高频共现规则牢牢记住，又可以让 deep 塔去补上未见过组合的泛化能力。顺着这个结构再往下看 DLRM，就会发现工业推荐的主流骨架开始把重心从“并联两路”转到“统一交互流程”。

DLRM: 把 embedding 交互整理成工业流水线

到了 DLRM，结构重心已经明显从“wide 分支是否保留”转向“所有特征在什么位置、以什么形式发生交互”。论文原图把这种变化画得很清楚，所以它特别适合拿来和前一页的 Wide & Deep 连着理解。



DLRM 总体结构。引自《Deep Learning Recommendation Model for Personalization and Recommendation Systems》Figure 1。

图中 dense features 和 sparse features 先分别完成底层表示：前者走 bottom MLP，后者通过 embedding lookup 得到向量，然后在中间的 interaction 模块里显式碰撞，最后再交给 top MLP 汇总。也就是说，交互不再是一条旁路能力，而是主干中的标准步骤。

这正是正文里把 DLRM 概括成“embedding + 显式交互 + top MLP”的原因。它的厉害之处并不只是效果，而在于输入编码、交互层和打分头的边界都非常清楚，既方便扩展大规模稀疏特征，也方便在真实广告与推荐系统里成为稳定、可复用的工业基线。

- 稀疏查表成本高；
- 明确需要字段级交互。

所以 DLRM 的厉害之处不只是效果，而是：

- 结构规整；
- 容易大规模训练；
- 容易做系统优化。

它的工业意义非常大，因为真实广告和推荐里确实有大量：

- 多字段离散特征;
- 超大 embedding 表;
- 大规模分布式训练和推理。

2.4 这条路线的优点

- 非常擅长处理多字段表格特征;
- 对高维离散 ID 类特征很强;
- 工业部署成熟;
- 与 CTR / CVR 排序任务天然贴合。

2.5 这条路线的不足

它虽然很会处理“当前这一行样本的字段组合”，但对“时间上的兴趣演化”处理得没那么自然。

比如下面两个用户，静态画像完全一样：

- 用户 A：最近连续看了 20 个母婴商品；
- 用户 B：最近连续看了 20 个健身器材。

如果只看静态画像，模型可能把这两个人当得很像。但实际上他们最近的兴趣完全不同。这就轮到第二条路线出场。

3 什么叫序列模型

3.1 还是用同一个电商例子

现在不只看用户静态画像，而是看他最近的行为序列：

$$h = [b_1, b_2, \dots, b_T].$$

例如：

- 第 1 次看了瑜伽垫；

- 第2次点了运动内衣；
- 第3次加购了蛋白粉；
- 第4次搜索了跑鞋。

这时候你会发现，只看“当前一行样本”已经不够了。因为用户的兴趣是带顺序的：

- 先看什么，后看什么，本身就是信息；
- 最近做过什么，往往比很久以前更重要；
- 当前候选和历史中的哪一段更相关，也会影响预测。

3.2 为什么序列这么重要

推荐系统里很多关键信号不是静态画像，而是“最近发生了什么”。

如果一个用户最近连续做了这些事：

- 搜索“婴儿车”；
- 浏览“奶瓶”；
- 收藏“纸尿裤”；
- 对“早教玩具”停留很久；

那这个用户此刻很可能处在一段明确的母婴兴趣窗口里。

这类信息很难只靠静态字段表达，却非常适合用序列模型去建模。

特征交互更像在问“这一刻有哪些字段组合值得加分”，序列模型更像在问“这个用户最近到底在经历什么兴趣过程”。

3.3 最简单的序列想法：把历史求平均

最朴素的想法是先把每个历史行为编码成向量 $\mathbf{e}_t$ ，再做平均池化：

$$\mathbf{h}_{\text{avg}} = \frac{1}{T} \sum_{t=1}^T \mathbf{e}_t.$$

这件事的好处很明显：

- 简单；
- 稳定；
- 推理开销低。

但问题也很明显：它默认所有历史一样重要。

也就是说：

$$\alpha_t = \frac{1}{T}, \quad \forall t.$$

这在很多场景里明显不合理。因为：

- 最近买过运动袜，比两个月前看过奶瓶更重要；
- 当前候选是跑鞋时，运动类历史应该被重点放大；
- 当前候选是奶瓶时，母婴历史才更关键。

换句话说，平均池化把：

- 两个月前看的東西；
- 五分钟前刚加购的东西；

一视同仁了。这显然不合理。

3.4 DIN: 让候选商品去“问”历史

DIN 的关键启发是：用户兴趣不是一个固定向量，它会随着“你问的是哪个候选 item”而变化。

我们可以把候选 item 的表示记成 $\mathbf{q}$ ，把历史行为的表示记成 $\mathbf{k}_t, \mathbf{v}_t$ 。DIN 会先计算当前候选和每条历史的相关性分数：

$$a_t = \text{MLP}([\mathbf{q}, \mathbf{k}_t, \mathbf{q} - \mathbf{k}_t, \mathbf{q} \odot \mathbf{k}_t])$$

或者用更简化的内积写法：

$$a_t = \mathbf{q}^\top \mathbf{k}_t.$$

然后把这些分数归一化为 attention 权重：

$$\alpha_t = \frac{\exp(\mathbf{q}^\top \mathbf{k}_t / \sqrt{d})}{\sum_{j=1}^T \exp(\mathbf{q}^\top \mathbf{k}_j / \sqrt{d})}.$$

最后做加权求和，得到面向当前候选 item 的兴趣表示：

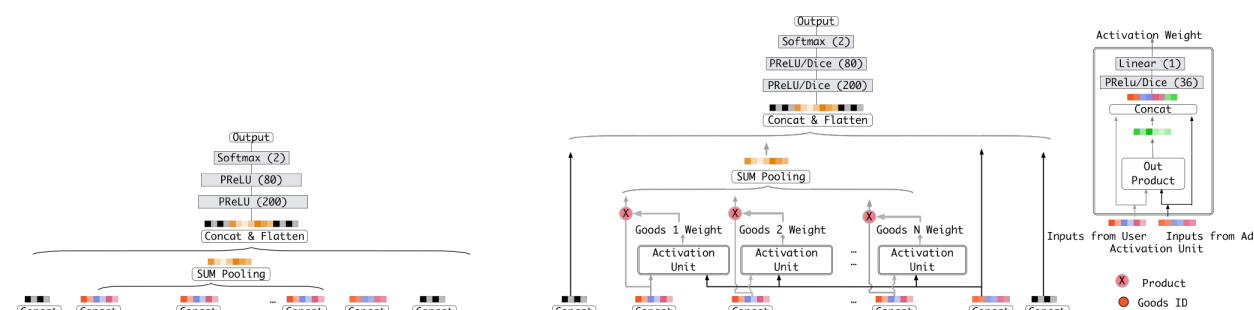
$$\mathbf{h}_{\text{seq}} = \sum_{t=1}^T \alpha_t \mathbf{v}_t.$$

DIN 最强的地方在于它改掉了平均池化的一个核心缺陷：

- 平均池化是“所有历史一视同仁”；

DIN: 候选商品如何重读历史行为

DIN 的转折点在于，它不再假设用户历史可以被压成一个固定向量。候选商品不同，历史里应该被强调的行为也应随之变化；而这一点只靠公式描述时往往抽象，结合原图就会直观很多。



DIN 局部激活网络结构。引自《Deep Interest Network for Click-Through Rate Prediction》Figure 2。

图的左半部分仍然是熟悉的 embedding 与 MLP，真正的变化出现在右侧的 local activation unit。候选 item 的表示会和每一个历史行为分别计算相关性，再把这些相关性转成加权汇聚权重，因此历史序列不会再被一视同仁地平均池化。

这样回头看正文里的注意力权重，就会更容易明白它们不是凭空出现的超参数，而是“当前候选去询问历史”这一动作的结构化结果。DIN 看起来仍然保留了 pooling 的外壳，但它在语义上已经从“平均看历史”改成了“带着问题读历史”。

- DIN 是“历史的重要性由当前候选决定”。

这意味着同一个用户，在面对不同候选 item 时，会得到不同的兴趣向量。

例如：

- 当前候选是跑鞋，模型会更关注运动相关历史；
- 当前候选是奶瓶，模型会更关注母婴相关历史。

DIN 的关键启发是：

- 用户兴趣不是一个固定向量；
- 它会随着“你问的是哪个候选 item”而变化。

3.5 DIEN: 不只看相关性，还看兴趣怎么演化

在 DIN 之前和同时期，另一类很重要的思路是 RNN / GRU 路线。它们的核心想法是：用户兴趣是按时间一步步更新的，后一时刻的兴趣状态应该继承前一时刻的信息。

最简化写法就是：

$$\mathbf{h}_t = \text{GRU}(\mathbf{e}_t, \mathbf{h}_{t-1}).$$

意思是：

- 第 t 个行为输入是 $\mathbf{e}_t$ ；
- 上一时刻兴趣状态是 $\mathbf{h}_{t-1}$ ；
- GRU 把二者融合成新的兴趣状态 $\mathbf{h}_t$ 。

如果把整个序列看完，最后就得到：

$$\mathbf{h}_T$$

它可以被看成“用户看到当前时刻之前的动态兴趣表示”。

相比平均池化，GRU 路线的优点是：

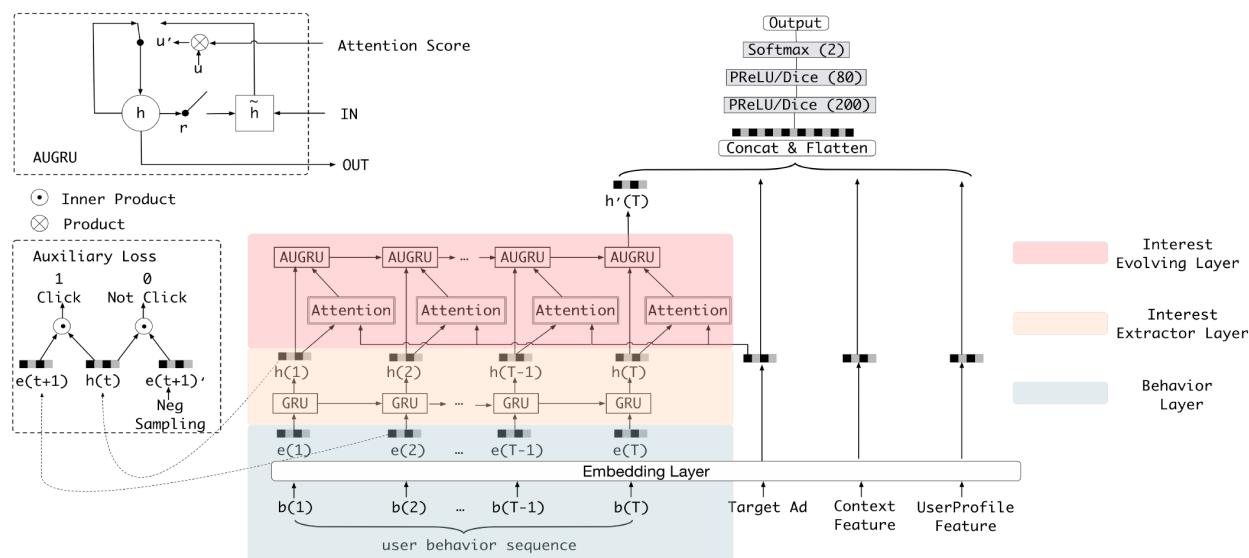
- 保留顺序信息；
- 能表达兴趣逐步变化；
- 能更自然地区分早期行为和近期行为。

但纯 GRU 路线也有一个问题：它虽然建模了顺序，却不一定知道“当前候选 item 最关心哪段历史”。

DIEN 在 DIN 基础上更进一步：

DIEN：先抽取兴趣，再建模兴趣演化

如果说 DIN 解决的是“当前候选该从历史里看什么”，那么 DIEN 进一步追问的是“这些兴趣本身是如何随时间形成并演化的”。论文原图把这两个层次拆得很清楚，也因此比单看文字更容易抓住它的核心增量。



DIEN 整体结构。引自《Deep Interest Evolution Network for Click-Through Rate Prediction》Figure 1。

图中底部行为序列先被编码成 embedding，然后由 interest extractor layer 抽取出逐时刻的兴趣状态；上层的 interest evolving layer 再让这些状态围绕目标 item 继续演化。左上角单独画出的 AUGRU 则说明，目标相关性并不是在最后一层才出现，而是直接进入了门控更新过程。

所以 DIEN 和 DIN 的差别，不能简单理解成“又多加了一层 attention”。它真正做的是先把兴趣轨迹显式建出来，再让轨迹朝当前候选继续收敛。正文里“先建轨迹，再看轨迹如何指向目标”的那句概括，对照这张图会更容易真正形成心智模型。

- 不只看“哪些历史与当前候选相关”；
- 还看“兴趣是怎么一步步演化来的”。

它大致可以分成两层。

第一层，先用 GRU 从原始行为序列里抽取兴趣状态：

$$\mathbf{h}_t = \text{GRU}(\mathbf{e}_t, \mathbf{h}_{t-1}).$$

第二层，再结合当前候选 item 对这条轨迹做目标感知的演化。可以用一种直觉写法表示：

$$a_t = \text{Attention}(\mathbf{q}, \mathbf{h}_t).$$

在 DIEN 里，这一步进一步通过 AUGRU 把 attention 融入更新门。它最核心的思想不是“简单再加一层 attention”，而是：

- 先抽出兴趣轨迹；
- 再让兴趣轨迹在目标 item 条件下继续演化。

这类模型特别适合：

- 用户兴趣在短期内明显切换；
- 一个候选 item 只和历史中的某一段相关；
- 当前转化受连续多步行为共同影响。

它相比 DIN 更强的直觉可以概括成：

- DIN 更像“从历史里挑重点”；
- DIEN 更像“先建轨迹，再看这条轨迹如何指向当前候选”。

3.6 BST / SASRec / BERT4Rec：用 Transformer 看序列

后来大家发现，Transformer 特别适合处理长行为序列。这类模型背后的共同逻辑是：

- 不再只把用户兴趣压缩成一个递推状态；
- 而是让序列中任意两个位置都可以直接相互作用。

首先，每个历史行为会先被编码成：

$$\mathbf{x}_t = \mathbf{e}_t + \mathbf{p}_t$$

其中：

- $\mathbf{e}_t$ 是 item 或行为 embedding；

- $\mathbf{p}_t$ 是位置编码。

然后把每个位置投影成 Query、Key、Value:

$$\mathbf{q}_t = \mathbf{W}_Q \mathbf{x}_t, \quad \mathbf{k}_t = \mathbf{W}_K \mathbf{x}_t, \quad \mathbf{v}_t = \mathbf{W}_V \mathbf{x}_t.$$

核心自注意力是:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d}} \right) V.$$

把它展开成单个位置的视角, 就是:

$$\text{score}_{t,j} = \frac{\mathbf{q}_t^\top \mathbf{k}_j}{\sqrt{d}}, \quad \alpha_{t,j} = \frac{\exp(\text{score}_{t,j})}{\sum_{\ell} \exp(\text{score}_{t,\ell})}, \quad \mathbf{o}_t = \sum_j \alpha_{t,j} \mathbf{v}_j.$$

这说明:

- 第 t 个位置最终读到的信息;
- 是全序列其他位置按权重加权后的总和。

这一大类方法里, 几篇代表工作解决的问题并不完全一样。

- BST 更贴近工业 CTR / 排序场景, 它通常会把用户行为序列、候选 item、其他上下文特征一起送入 Transformer 风格模块, 然后输出排序分。
- SASRec 更偏“下一物品预测”范式, 它使用单向自注意力, 也就是当前位置只能看自己之前的历史, 不能偷看未来。
- BERT4Rec 借鉴的是 BERT 的 masked language modeling 思想。它不是只看单向历史, 而是把序列中某些位置 mask 掉, 让模型根据双向上下文恢复它们:

$$\mathcal{L}_{\text{mask}} = - \sum_{t \in \mathcal{M}} \log P(b_t | h_{\setminus t}).$$

如果粗略区分:

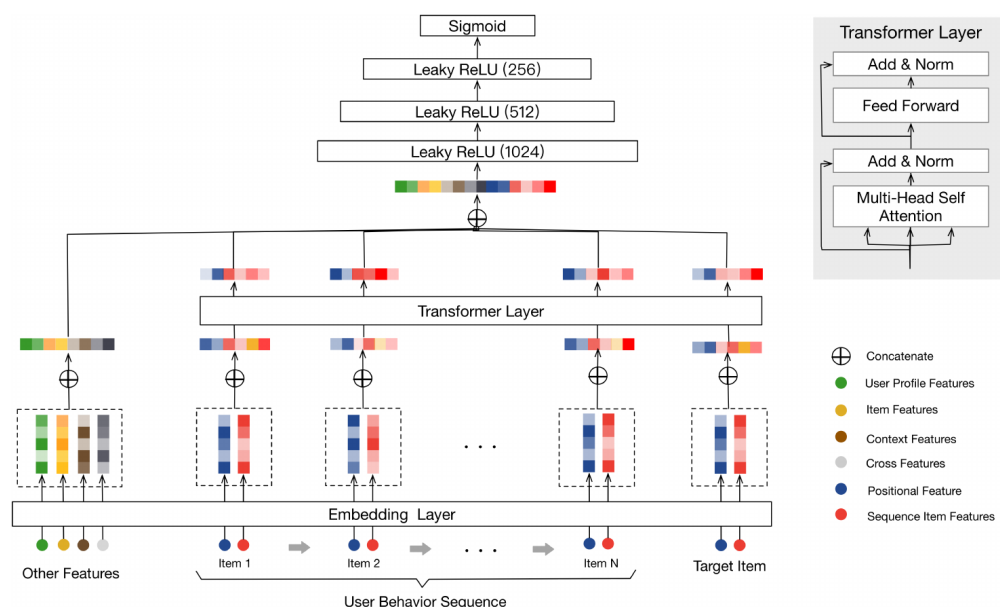
- BST 更贴近工业 CTR / 排序;
- SASRec 更像自注意力版 next-item 序列推荐;
- BERT4Rec 更像双向序列表示学习。

这一类方法之所以近几年影响非常大, 是因为它们把序列推荐从:

- “一步步传状态”

BST: 把行为序列、目标 item 和上下文一起送入 Transformer

BST 进入工业排序场景之后，Transformer 不再只负责对一条纯序列做编码，而是开始和目标 item、上下文特征一起参与最终打分。这个变化如果只靠概念描述容易被忽略，但原图把它画得非常具体。



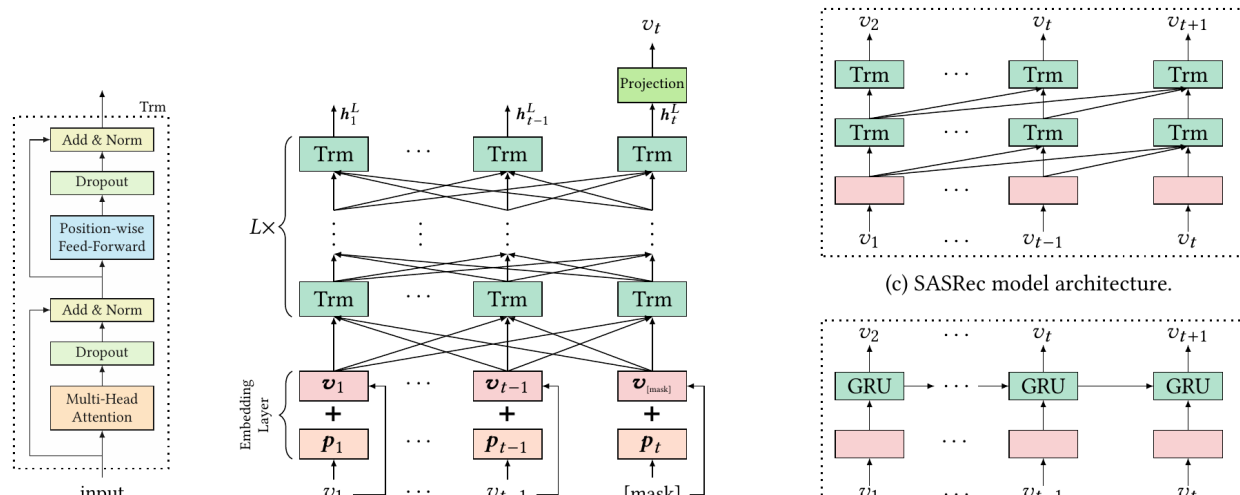
BST 总体结构。引自《Behavior Sequence Transformer for E-commerce Recommendation in Alibaba》Figure 1。

图里行为序列先做 embedding，再进入 self-attention 模块；与此同时，目标 item 与其他特征并没有被排除在外，而是在后续一起进入聚合与 MLP 打分阶段。也就是说，BST 不是把 Transformer 当成一个孤立的序列编码器，而是把它放回了 CTR / ranking 的联合建模链路里。

因此，BST 最适合被理解成“工业排序里的 Transformer 序列模块”。它关心的不只是下一步行为预测，而是历史序列如何与候选和上下文共同决定最终分数。带着这个视角再看下一页的 BERT4Rec，对两类 Transformer 序列建模的分工会更容易区分开来。

BERT4Rec: 双向编码为什么会改变序列建模方式

BERT4Rec 这张对照图的价值，在于它不是只画出自己的结构，而是把 RNN、SASRec 和双向 Transformer 的差别一次性摆在了一起。对初学者来说，这比单独看任何一个模型都更容易建立比较坐标。



BERT4Rec 与 SASRec、RNN 的结构对照。引自《BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer》Figure 1。

左侧 RNN 只能沿时间顺序递推，中间的 SASRec 虽然换成了 self-attention，却仍然保持单向读取；右侧 BERT4Rec 则用 masked item prediction 让当前位置同时利用左右文脉。正文里提到的“单向”和“双向”，在这张图里不再是抽象标签，而是明确体现在信息能否从未来位置回流到当前表示上。

因此，BST 与 BERT4Rec 虽然都属于 Transformer 路线，但关注点并不相同。前者更贴近工业排序里的联合打分，后者更像双向序列表示学习。把这张图和正文关于 SASRec 的描述对照起来看，三者的边界会比纯文字说明清楚得多。

推进到了：

- “序列中任意位置都能直接建模关系”。

3.7 这条路线的优点

- 擅长建模近期兴趣和兴趣演化；
- 擅长利用历史点击、曝光、加购、购买等行为序列；
- 更容易捕捉“此刻”用户想要什么。

3.8 这条路线的不足

如果你只盯着序列，也会丢很多东西。比如：

- 用户城市；

- 设备类型;
- 广告位;
- 商家属性;
- 价格带;
- 供给侧的多字段属性。

这些很多不是“时间序列”，但它们对 CTR / CVR 非常重要。也就是说：

- 序列模型很会看“过程”;
- 但不一定很会处理“多字段表格结构”。

这就解释了为什么两条路线会长期并存。

4 什么叫“两条路线单独演进”

这是最容易把人绕晕的一句话，但本质上不复杂。它不是说这两类方法完全没有交流，而是说：很多年里，工业系统通常不是用一套统一骨干模型同时处理序列和多字段特征，而是各自做一支，再在后面拼起来。

4.1 一个典型旧架构长这样

非序列特征

-> embedding / MLP / FM / DLRM 分支
-> 得到 z_feat

行为序列

-> DIN / GRU / Transformer 分支
-> 得到 z_seq

最后

```
-> concat([z_feat, z_seq])
```

```
-> MLP
```

```
-> score
```

你可以把它理解成：

- 左边一支是表格特征专家；
- 右边一支是序列专家；
- 最后两位专家把意见写在一起，交给一个总裁决策层。

4.2 为什么会这样

因为历史上这两类问题本来就长得不一样。

特征交互路线更像表格学习：

$$\mathbf{x}_{\text{tab}} = [x_1, x_2, \dots, x_m]$$

重点是字段之间的组合关系：

$$x_i x_j, \quad x_i x_j x_k.$$

序列路线更像时序建模：

$$\mathbf{x}_{\text{seq}} = [\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_T]$$

重点是：

- 顺序；
- 最近性；
- 长短期依赖；
- 兴趣转移。

因为问题形式不同，所以：

- 数据组织不同；
- 模型模块不同；
- 训练技巧不同；
- 工程系统不同。

久而久之，它们就各自长成了成熟体系。

4.3 “单独演进”具体体现在哪

4.3.1 输入表达分裂

一部分输入被当成表格字段：

- 用户画像；
- 物品属性；
- 上下文。

另一部分被当成行为序列：

- 点击历史；
- 曝光历史；
- 转化历史。

两边不是同一套 token，也不是同一套编码逻辑。

4.3.2 模型结构分裂

一边是：

- FM；
- DCN；
- Wide & Deep；
- DLRM。

另一边是：

- DIN；
- DIEN；
- GRU4Rec；
- SASRec；
- BST。

4.3.3 目标与阶段分裂

工业系统里常见两阶段甚至多阶段流程：

召回 → 粗排 → 精排 → 重排。

序列模型常常在召回阶段做用户兴趣表征，或在排序阶段做历史行为编码；而特征交互模型常常在 CTR / CVR 排序阶段做主力打分。于是“谁负责什么”长期是分开的。

4.3.4 工程与硬件分裂

一边重点是：

- 超大 embedding table;
- 稀疏查表;
- 多字段并行。

另一边重点是：

- 长序列截断;
- attention;
- cache;
- padding 和 mask。

所以系统越来越像两套引擎硬拼。

4.4 这种分离为什么会成为瓶颈

第一，跨范式交互太浅。旧架构里经常是先把序列压成一个向量 $\mathbf{z}_{\text{seq}}$ ，再和表格特征向量 $\mathbf{z}_{\text{feat}}$ 拼接：

$$\mathbf{z} = [\mathbf{z}_{\text{feat}}; \mathbf{z}_{\text{seq}}], \quad s = g(\mathbf{z}).$$

问题在于：

- 表格特征很难深入影响“该看历史里的哪一步”;
- 序列信息也很难细粒度地反作用于其他字段交互。

第二，优化目标不完全一致。上游觉得好的用户表示，下游不一定最喜欢；召回与排序衡量“相关”的方式也不一定一致。

第三，扩展性差。序列越来越长，字段越来越多，模型越来越大，时延越来越紧，如果还是两套结构平行堆上去，整体成本会非常高。

第四，工程复杂度高。你不只是多了一个模块，而是多了一整套：

- 数据预处理;
- 特征 schema;
- 序列切分逻辑;
- 模型导出;
- 推理服务;
- 时延优化。

最终很容易变成：效果提升一点点，工程复杂度上升很多。

5 统一建模到底在统一什么

现在进入真正的核心。

统一建模最朴素的一句话是：

不再把“非序列特征”和“行为序列”当成两条独立支路，而是把它们放进同一个表示空间、同一套骨干模块里联合建模。

5.1 先搭一个统一视角：推荐排序的输入到底长什么样

我们先把一条样本统一写成：

$$\mathbf{x} = (NS, S)$$

其中：

- NS : non-sequential features, 非序列特征，比如用户画像、候选 item、上下文、交叉特征；
- S : sequential features, 用户历史行为序列，可以是一条或多条、多域序列。

排序模型输出一个分数：

$$\hat{y} = f_{\theta}(NS, S).$$

如果是点式二分类排序，训练目标通常还是 BCE：

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{n=1}^N \left[y_n \log \hat{y}_n + (1 - y_n) \log(1 - \hat{y}_n) \right].$$

OneTrans 和 HyFormer 的核心区别，不在于最后 loss 完全不同，而在于：它们把 NS 和 S 怎么组织、怎么流过主干、怎么互相作用，设计得完全不同。

5.2 统一可以发生在四层

5.2.1 第一层统一：输入表示统一

这层统一的核心是：不再把一部分输入当表格，另一部分输入当另一套特殊对象，而是尽量都转成统一 token 或统一向量单元。

例如：

- user_city=shanghai
- item_brand=nike
- context_hour=22

- hist_click_item=running_shoe

都可以变成 token。

如果写成一般形式：

$$\mathbf{e}_k = E_{\text{value}}(x_k) + E_{\text{field}}(f_k) + E_{\text{type}}(t_k) + E_{\text{meta}}(m_k),$$

它其实在做四件事：

- 值本身的表示；
- 字段身份；
- 类型身份；
- 附加元信息，比如位置、行为域、时间桶。

5.2.2 第二层统一：交互机制统一

旧系统经常是：

- 表格特征走 MLP / FM；
- 序列走 RNN / attention。

统一建模想做的是：尽量让它们在同一类 block 中交互。例如统一用 Transformer block：

$$H^{(\ell+1)} = \text{TransformerBlock}(H^{(\ell)}).$$

或者统一用某种 mixer / hybrid block。这样做的意义是：

- 序列 token 和非序列 token 可以在同一层直接交互；
- 而不是最后才拼起来。

5.2.3 第三层统一：目标统一

这层统一讲的是：模型内部所有输入最终都是为了同一个打分目标服务。

$$s = f_{\theta}(u, i, c, h).$$

如果整套骨干都是围绕这个最终打分训练，那么用户静态特征、item 特征、上下文、历史行为都会被拉到同一个目标上联合优化。这比“一个分支先做自己的目标，再和主任务拼接”更一致。

5.2.4 第四层统一：部署路径统一

这一层特别工业，但很重要。真正有价值的统一模型，通常不仅是论文上统一，还要在工程上统一：

- 一套输入 schema；
- 一套模型图；
- 一条推理链路；
- 一套时延优化逻辑。

5.3 为什么统一建模更自然

因为真实推荐任务本来就不是两个独立问题。它从来都是一个整体问题：

这个用户在这个时刻、看到这个候选 item、带着这段历史和这些上下文时，是否会发生目标行为？

统一建模就是让模型从一开始就把这些信息视为同一件事。

5.4 统一建模不等于“把所有东西粗暴拼接”

这点非常重要。真正的统一建模不是：

- 把向量简单 concat 一下。

真正的统一建模是：

- 在编码层统一表示；
- 在 backbone 层统一交互；
- 在训练和推理层统一优化。

否则就只是“多加了一些输入”，不是统一范式。

6 OneTrans: 把序列 token 和非序列 token 放进同一条 Transformer 流

6.1 OneTrans 最核心的想法

OneTrans 的想法很激进，但也很干脆：

既然大模型已经证明“统一 token 序列 + 统一 Transformer backbone”是一套很好 scale 的范式，那推荐排序是不是也可以把异构输入都尽量 token 化，然后放进同一个 backbone？

它的统一，不是最后拼一下，而是从一开始就统一。

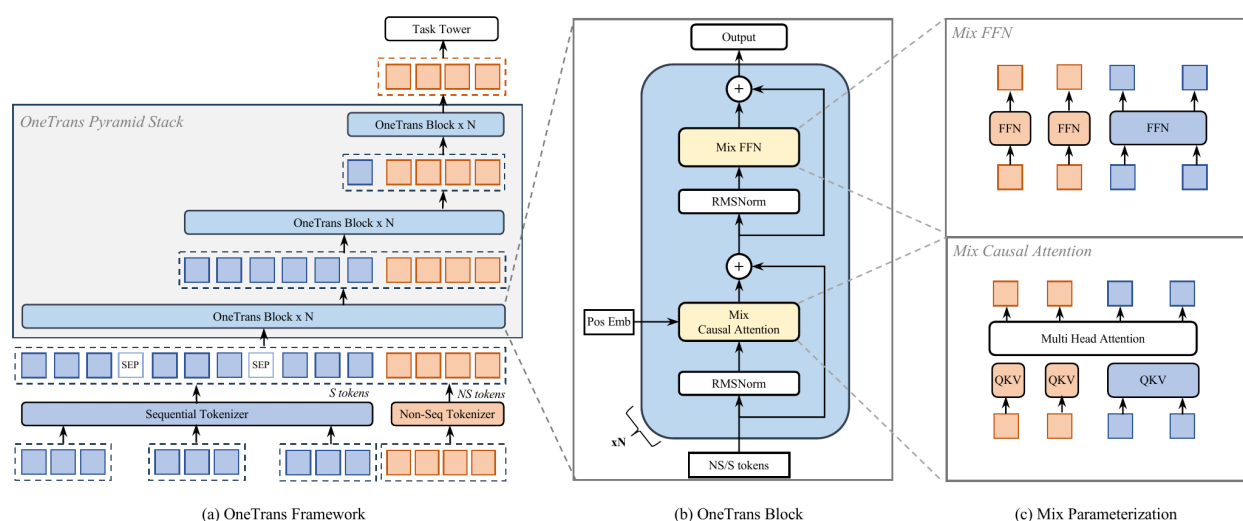
6.2 第一步：统一 token 化

OneTrans 先把输入拆成两类 token：

- *S*-tokens：序列 token，对应用户多行为序列；

OneTrans: 统一 token 化之后, 主干里到底发生了什么

OneTrans 的统一并不是把一个序列编码器和一个特征交互模块简单串起来, 而是从 token 视角直接改写了输入组织方式。论文系统图把这种“先统一输入, 再统一主干”的思想浓缩得非常完整。



OneTrans 系统结构图。引自《OneTrans: Unified Feature Interaction and Sequence Modeling with One Transformer in Industrial Recommender》Figure 2。

图左侧先把序列特征和非序列特征分别 token 化, 再通过 pyramid stack 逐层压缩长序列长度; 中间的统一 block 说明二者随后进入同一条 causal Transformer 主干; 右侧 mixed parameterization 则进一步强调, 统一不等于所有 token 完全同参, 而是在共享主干下保留异构 token 的差异化处理。

这样回头看正文 6.2 到 6.6, 就会更容易明白 OneTrans 的重点不只是“同流”两个字, 而是如何在同一条信息流里同时容纳候选感知读取、异构特征交互和工业侧的算力约束。这张图几乎把 unified tokenizer、causal mask、mixed FFN 和 pyramid stack 的关系一次性讲清楚了。

- NS -tokens: 非序列 token, 对应 user / item / context / cross 等非序列特征。

于是初始输入可以写成:

$$X^{(0)} = [S\text{-tokens}; NS\text{-tokens}] \in \mathbb{R}^{(L_S + L_{NS}) \times d}.$$

这里:

- L_S 是序列 token 总长度;
- L_{NS} 是非序列 token 个数;
- d 是统一 hidden dimension。

你可以把它想成一串长 token:

[历史行为 1] [历史行为 2] ... [历史行为 T] [SEP]
 [用户画像 token] [候选 item token] [上下文 token] ...

6.2.1 非序列 token 是怎么来的

论文里对非序列特征提供了两种 tokenizer 思路：

1. Group-wise tokenizer: 按语义分组后分别投影；
2. Auto-split tokenizer: 把所有非序列特征 concat 后过一个 MLP，再切成多个 token。

如果写成教学化形式，可以理解为：

$$NS\text{-tokens} = \text{Tokenizer}_{NS}(NS).$$

它背后的直觉是：

- group-wise 更保留结构先验；
- auto-split 更像“大一统切块”，算子更简单。

6.2.2 序列 token 是怎么来的

OneTrans 接受多行为序列，例如 click / cart / buy 等不同序列。对第 i 条行为序列：

$$S_i = [\mathbf{e}_{i1}, \mathbf{e}_{i2}, \dots, \mathbf{e}_{iL_i}],$$

每个事件 embedding $\mathbf{e}_{ij}$ 可以由 item ID 和 side information 拼接后再投影得到。教学化地写：

$$\tilde{S}_i = [\text{MLP}_i(\mathbf{e}_{i1}), \dots, \text{MLP}_i(\mathbf{e}_{iL_i})] \in \mathbb{R}^{L_i \times d}.$$

然后多条序列要合并成一条统一的 S -token 流。OneTrans 给了两种策略：

1. timestamp-aware: 按真实时间交织;
2. timestamp-agnostic: 按行为意图强弱排序, 例如 purchase -> cart -> click, 再用 [SEP] 隔开。

6.3 第二步: 放进同一个 causal Transformer block

OneTrans 的 block 结构看起来很像标准 pre-norm Transformer:

$$Z^{(n)} = \text{MixedMHA}(\text{Norm}(X^{(n-1)})) + X^{(n-1)},$$

$$X^{(n)} = \text{MixedFFN}(\text{Norm}(Z^{(n)})) + Z^{(n)}.$$

但它的关键不是“用了 Transformer”这么简单, 而是:

它让 S -token 和 NS -token 在同一张 causal mask 下共同流动。

这意味着在同一个 stack 内部, 可以同时发生:

- 序列内部的因果建模;
- 跨序列交互;
- user / item / context 之间的高阶交互;
- 非序列 token 对历史序列的 target-aware 聚合。

6.4 第三步: Mixed parameterization

OneTrans 认为推荐里的 token 并不都像文本 token 一样同质。

- 序列 token 往往比较同构: 都表示“行为事件”;
- 非序列 token 则异质性很强: 用户年龄、item ID、上下文位置、设备类型, 语义跨度很大。

因此它没有对所有 token 完全共用一套参数, 而是采用 mixed parameterization:

- 所有 S -token 共用一套 $Q/K/V$ 投影和 FFN;
- 每个 NS -token 使用 token-specific 的 $Q/K/V$ 投影和 FFN。

教学化写成:

$$(\mathbf{q}_i, \mathbf{k}_i, \mathbf{v}_i) = (\mathbf{W}_i^Q \mathbf{x}_i, \mathbf{W}_i^K \mathbf{x}_i, \mathbf{W}_i^V \mathbf{x}_i),$$

其中

$$\mathbf{W}_i^\Psi = \begin{cases} \mathbf{W}_S^\Psi, & i \leq L_S, \\ \mathbf{W}_{NS,i}^\Psi, & i > L_S, \end{cases} \quad \Psi \in \{Q, K, V\}.$$

FFN 也类似: S 侧共享, NS 侧 token-specific。

这一步为什么重要? 因为 OneTrans 并不想粗暴地说:

“既然统一建模, 就所有 token 都必须一套参数到底。”

它的判断更工程化:

- 统一的是主干流和交互空间;
- 但对推荐里异质性很强的非序列 token, 仍保留局部参数自由度。

6.5 第四步: causal mask 到底在统一什么

OneTrans 使用 standard causal attention mask, 而且把 NS -token 放在 S -token 之后。这样会带来三个很关键的效果。

6.5.1 S-side: 序列 token 只看过去

对 S -token 而言, 它仍然是因果序列建模:

- 如果按 timestamp-aware 合并, 就是真正的时序依赖;
- 如果按行为意图排序, 则高意图行为更靠前, 可“过滤”后面较弱信号。

6.5.2 NS-side: 非序列 token 能看完整序列历史

由于 NS -token 排在 S -token 后面, 所以它们在 causal mask 下可以看到整个 S -token 历史。这其实非常像一种统一版 target attention:

- 候选 item token、上下文 token、用户静态画像 token;
- 都可以在同一注意力机制里对整段历史做聚合。

换句话说, OneTrans 不是单独做一个 DIN attention, 而是把这种“候选感知地读历史”的动作塞进统一 backbone 里。

6.5.3 mask 天然支持 pyramid compression

因为 causal 信息会越来越向后部 token 聚集, 所以后面的 token 往往更“汇总”。这为后面的 pyramid stack 做了铺垫。

6.6 第五步: Pyramid Stack

如果把所有序列 token 全程保留, 统一 Transformer 的算力会非常贵。所以 OneTrans 做了一个很关键的工程设计: 每层只让最近的一部分 S -token 发 query, 但仍让它们对完整 keys / values 做 attention。

教学化地说，在某层里只保留尾部索引集 $\mathcal{Q}$ 发起 query:

$$\mathbf{q}_i = \mathbf{W}_i^Q \mathbf{x}_i, \quad i \in \mathcal{Q},$$

但 key / value 仍来自全序列。算完之后，只保留这些尾部输出，序列长度从 L 缩到 L' ，继续上更深层。

你可以把它理解成:

- 底层: 看更长、更细的历史;
- 高层: 只保留更浓缩、更靠后的代表性 token;
- 最上层: S -token 数量缩到接近 NS -token 数量。

6.7 第六步: Cross-request / Cross-candidate KV caching

OneTrans 的另一个工业味很重的点，是它特别强调 KV caching。

在推荐 serving 里，同一个 request 通常会对应多个 candidate item:

- 用户历史序列不变;
- user-side token 大多不变;
- 变化的只是候选 item 相关 NS -token。

因此 OneTrans 可以缓存大量 S -side 的 key / value，避免每个 candidate 都重新算一次。而且如果下一次 request 到来时用户历史只是 append-only 地增加了一点点，那么就只需要对新增的 ΔL 行为做增量更新。

6.8 OneTrans 到底统一在哪

现在可以给出一句更精确的话:

OneTrans 统一的是“token 视角下的输入组织”和“同一条 causal Transformer 主干里的信息流”。

它不是“先序列、后交互”，而是:

- 统一 token 化;
- 统一 causal mask;
- 统一 block 堆叠;
- 再用 mixed parameterization 和 pyramid 保住工程可行性。

6.9 OneTrans 的优势与代价

6.9.1 优势

1. 架构极简，概念非常统一;

2. 很容易继承 Transformer / LLM 的系统优化;
3. 非序列 token 可以很早就影响序列侧聚合;
4. 适合做“大 backbone 统一 scaling”。

6.9.2 代价

1. 真正全统一后, 序列很长时计算压力仍然不小;
2. token 全混在同一条流里, 对不同序列的独特语义边界要求更高;
3. 对工业推荐里异构特征非常强的场景, 单纯 self-attention 不一定是省算或最优交互方式。

7 HyFormer: 不是把所有 token 硬塞一条流, 而是用 Global Query 做统一接口

7.1 HyFormer 的出发点

HyFormer 的出发点与 OneTrans 很像:

- 也认为“先长序列压缩、再 late fusion”的两阶段流水线不够好;
- 也想让序列建模与特征交互更早、更深地联动。

但 HyFormer 的判断和 OneTrans 不一样:

在工业大规模排序里, 并不是所有统一都应该通过“所有 token 一起 self-attend”来做。

HyFormer 认为更现实的统一路径是:

- 让长序列仍以比较高效、比较可控的方式编码;
- 让异构非序列特征先生成一组 semantic queries / global tokens;
- 再用这些 query 去不断读取序列、再混合异构特征。

所以 HyFormer 更像一种 hybrid unified backbone。

7.2 HyFormer 的总体图景

你可以把 HyFormer 的每一层理解成两步交替:

Global Queries

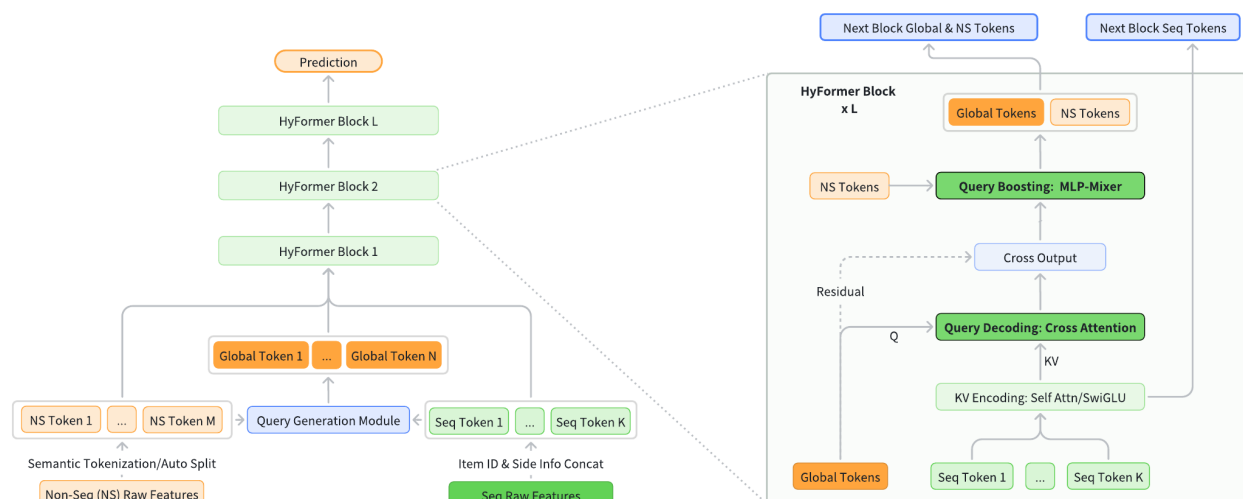
- > Query Decoding
- > 从长序列 K / V 里读出与当前 query 相关的信息
- > 得到 sequence-aware queries

sequence-aware queries + 非序列 token

- > Query Boosting
- > 做轻量 token mixing / feature interaction
- > 得到更强的 queries

HyFormer: Global Query 如何在两侧之间往返

HyFormer 选择了另一条统一路径。它并不要求所有 token 一上来就做同流 self-attention, 而是把 global query 当成序列侧与异构特征侧之间的统一接口, 这也是它和 OneTrans 最本质的结构差异。



HyFormer 总体结构图。引自《HyFormer: Revisiting the Roles of Sequence Modeling and Feature Interaction in CTR Prediction》Figure 1。

图左侧先根据非序列特征生成一组 global queries; 随后每一层都交替执行 query decoding 和 query boosting, 让 queries 先去读取长序列, 再把读回来的信息与异构特征重新融合。也正因为这种往返读取发生在每一层, query 会逐步从“懂静态特征”变成“同时懂序列和异构交互”的统一接口。

因此, HyFormer 的统一发生在接口和层间交互机制上, 而不是强制所有输入完全同构。这和正文里反复强调的判断完全一致: 它更像一个带有明确模块边界的 hybrid unified backbone, 用层级式的信息交换来代替 OneTrans 那种更彻底的同流自注意力统一。

再进入下一层

所以 HyFormer 的“统一”, 不是所有 token 都进同一条自注意力流, 而是:

每一层都在重复“先问长序列, 再和异构特征混合”这个循环。

7.3 第一步: Query Generation

HyFormer 先把非序列特征做 semantic grouping, 再生成一组 query token。论文里一个非常关键的思想是:

- query 不是只来自 candidate item;
- 而是来自 non-sequential features 加上 pooled sequence summary 的联合信息。

教学化写法:

$$\text{GlobalInfo} = \text{Concat}(F_1, F_2, \dots, F_M, \text{MeanPool}(\text{Seq})),$$

然后生成 N 个 query:

$$Q^{(0)} = [\text{FFN}_1(\text{GlobalInfo}), \dots, \text{FFN}_N(\text{GlobalInfo})] \in \mathbb{R}^{N \times D}.$$

这一步很像在做:

- 给当前样本生成一组语义探针;
- 它们携带候选 item、上下文、用户静态画像, 以及序列全局摘要;
- 后面这些探针会拿着这组全局语义去 interrogate 长序列。

7.4 第二步: Sequence Representation Encoding

HyFormer 对长序列并不是只有一种编码方法, 它给了三档 capacity-efficiency trade-off:

1. Full Transformer Encoding: 容量最高;
2. LONGER-style Efficient Encoding: 用短 query 去 cross-attend 全历史;
3. Decoder-style Lightweight Encoding: 更轻、更省时延。

因此, 对第 ℓ 层序列表示, 可以抽象成:

$$H^{(\ell)} = \text{SeqEncoder}^{(\ell)}(S),$$

再线性投影成 key / value:

$$K^{(\ell)} = H^{(\ell)} W_K^{(\ell)}, \quad V^{(\ell)} = H^{(\ell)} W_V^{(\ell)}.$$

这里跟 OneTrans 的根本区别是:

- OneTrans 更像 “token 一进主干就开始统一 self-attend”;
- HyFormer 更像 “长序列一侧仍保持相对独立的 sequence representation encoding, 但它不是编码完就彻底冻结, 而是每层都重新产出 layer-wise K / V, 供 query 去读取”。

所以 HyFormer 不是完全分离, 也不是完全合并, 而是: 用 layer-wise K / V 把长序列侧持续接进统一主干循环。

7.5 第三步: Query Decoding

这一步是 HyFormer 最像 “统一建模” 的核心接口。

对于第 ℓ 层, query 去 cross-attend 当前层的长序列 K / V:

$$\tilde{Q}^{(\ell)} = \text{CrossAttn}(Q^{(\ell-1)}, K^{(\ell)}, V^{(\ell)}).$$

这一步到底在干什么? 可以把它想成:

- query 本身带着 “当前推荐任务到底在意什么” 的全局语义;
- sequence K / V 则携带 “长序列里具体发生过什么”;
- cross-attention 把二者对齐后, 输出 “已经读过序列” 的 query。

所以 Query Decoding 不是一般的序列压缩, 而是: 让异构特征形成的 query 主动去读长序列。

7.6 第四步: Query Boosting

经过 Query Decoding 后, query 已经有了 sequence-aware 信息, 但还不够。因为它只是 “读过了序列”, 还没有充分和所有非序列 token 做异构交互。

于是 HyFormer 在每层接着做 Query Boosting。论文里做法偏向 MLP-Mixer / RankMixer 风格的 token mixing。教学化写法可以理解成:

先把 decoded queries 与非序列特征拼起来:

$$Q_{\text{all}}^{(\ell)} = [\tilde{Q}^{(\ell)}; F_1; \dots; F_M].$$

然后做轻量 token mixing 和 per-token FFN:

$$Q_{\text{boost}}^{(\ell)} = \text{Boost}(Q_{\text{all}}^{(\ell)}) + Q_{\text{all}}^{(\ell)}.$$

这一步为什么叫 boosting? 因为它不是重新做一大套 sequence attention, 而是:

- 在 query-level 把不同 query 之间、query 与非序列 token 之间再 “推一把”;

- 让下一层 query 更有表达力。

7.7 第五步: 逐层交替, 形成 alternating optimization 风格的统一骨干

HyFormer 的一层就是:

1. Query Decoding;
2. Query Boosting。

堆很多层以后, 就形成一种交替循环:

问历史 -> 混异构特征 -> 再拿更强的 query 去问历史 -> 再混异构特征

这就是论文里强调的 alternating optimization process 的直觉。它不是把统一建模理解成“一次性揉在一起”, 而是理解成: 让 query 在层间不断往返于“长序列读取”和“异构特征交互”之间。

7.8 HyFormer 为什么不把多条序列都拼成一条大流

HyFormer 还有一个很关键的设计选择: 每条行为序列在每个 block 里独立处理。

也就是说:

- 对每条 sequence, 都有专门的 query token 去做 Query Decoding;
- 不强行把所有 sequence merge 成一条统一大 token 流;
- 跨序列信息更多是在 query-level 的 boosting 阶段去做。

为什么这么做? 论文给出的核心判断是:

- 不同序列往往语义不同;
- 强行 merge 会迫使不同 sequence 共享一套 global tokens;
- 表示可能不够有区分度;
- 还会牺牲 serving 效率。

所以 HyFormer 的统一, 不是“强制同流”, 而是: 保持 sequence-specific semantics, 再在 query-level 统一交互。

7.9 HyFormer 的工程优化

HyFormer 论文里还花了不少篇幅讨论工程效率, 这很关键, 因为它不是只谈模型想法。

7.9.1 GPU Pooling for Long Sequence

长序列特征往往非常大, Host-to-Device 传输和 host memory 都很吃紧。HyFormer 观察到:

- 序列里的 truly unique feature id 远少于 token 总数;
- 经验上, unique id 常常只占总 token 的一小部分。

因此它会先做稀疏去重和压缩，再在 GPU 侧重建原始 sequence feature，从而减少传输和 host memory 压力。

7.9.2 Asynchronous AllReduce

为了减少同步通信带来的空转，它把 step k 的梯度同步与 step $k + 1$ 的前反向重叠，代价是 dense 参数会有一步 staleness。这是一种很典型的“大规模工业训练吞吐优先”设计。

7.10 HyFormer 到底统一在哪

现在可以给一句更准确的描述：

HyFormer 统一的是“长序列读取接口”和“异构特征交互接口”，通过 global query 在每一层里反复把两边绑在一起。

它不是一条纯 self-attention 流，而是：

- query generation;
- layer-wise sequence K / V;
- cross-attention decoding;
- token-mixing boosting;
- 多层交替。

7.11 HyFormer 的优势与代价

7.11.1 优势

1. 比“全 token 同流 self-attend”更节制，尤其适合长序列；
2. 保留了 sequence-specific 结构，不必强行 merge 多序列；
3. Query Decoding 与 Query Boosting 分工清楚，容易解释；
4. 更容易在“序列侧容量”和“交互侧容量”之间分开调节。

7.11.2 代价

1. 架构比 OneTrans 更复杂，模块边界更多；
2. 它不是最纯粹的一体化 Transformer，系统实现更像“混合主干”；
3. 想真正调到最好，需要同时兼顾 query 数量、sequence encoder 方案、boosting 宽度等多个旋钮。

8 OneTrans 和 HyFormer 的核心差异，逐点掰开说

8.1 一张对照表先看清楚

维度	OneTrans	HyFormer
统一切入点	输入 token 化加单一 Transformer 主干	Global query 接口加交替式层内循环
主干风格	单一 causal Transformer stack	Hybrid backbone: Query Decoding + Query Boosting
长序列如何接入	直接作为 S -token 流的一部分	先编码成 layer-wise K / V ，再供 query 读取
非序列特征如何接入	作为 NS -token，与 S -token 同流	参与 query generation，并在 boosting 阶段继续交互
多序列处理	尽量统一进一条 token 流	每条 sequence 独立 decoding，后面 query-level 汇合
更像哪种哲学	端到端统一骨干	接口统一、机制分工明确

8.2 信息流到底怎么走

8.2.1 OneTrans 的信息流

统一 token 序列

-> mixed causal attention

-> mixed FFN

-> pyramid 压缩

-> head

也就是说，它更像：所有东西一开始就在一个大房间里说话。

8.2.2 HyFormer 的信息流

非序列特征 -> 生成 query

长序列 -> 产出当前层 K / V

query 去读 K / V -> 得到 decoded query

decoded query 与非序列 token 再混合 -> boosted query

再进入下一层

它更像：先派一组代表去采访长序列，再回来开会，再派更强的代表继续采访。

8.3 为什么 HyFormer 不满意 OneTrans 这类“全流自注意力统一”

HyFormer 对这类 unified-block 模型的批评，大意可以总结成两点：

1. 如果把 global token 和 sequence token 都作为 key，而只让 global token 做 query，那么 global token 很容易互相注意自己，未必足够深入地读到 sequence item 的细节；
2. 直接用 self-attention 处理 feature interaction，在工业 feature interaction 模块上未必是最省

算、最稳的选择。

这不一定是放之四海而皆准的结论，但至少说明：

HyFormer 追求的是“更节制、更明确的统一接口”，而不是“能合就都合进 self-attention”。

8.4 为什么 OneTrans 不想走 HyFormer 那么“模块化”的路

反过来看，OneTrans 的思路则是：

1. 一旦把 sequence modeling 和 feature interaction 放在同一 Transformer stack，双向信息流天然就更充分；
2. 统一主干后，LLM 时代成熟的系统优化可以直接复用；
3. 架构越整齐，越容易沿着 depth / width 直接 scaling。

所以 OneTrans 的哲学更像：

与其设计很多“专门接口”，不如尽可能让 token 在一条主干里直接交流，再用 mixed parameterization 和 pyramid 把工程问题压住。

9 用一个具体样本来理解两者的差别

假设当前排序样本是：

- 用户：上海、iPhone、长期偏运动；
- 当前候选：Nike 跑鞋；
- 上下文：晚上 10 点，首页第 3 位；
- 序列：最近看过瑜伽垫、运动内衣、蛋白粉、跑步机，也看过一点母婴商品。

9.1 OneTrans 会怎么做

OneTrans 更像把它写成一串统一 token：

[历史：瑜伽垫] [历史：运动内衣] [历史：蛋白粉] [历史：跑步机] [历史：母婴商品] [SEP]
[用户城市 = 上海] [设备 = iPhone] [item=Nike 跑鞋] [时间 = 22 点] [slot=3] ...

然后在同一条 causal Transformer 里完成：

- 历史之间的顺序建模；
- item token 对历史的聚合；
- 用户静态画像与候选、上下文之间的交互；
- 不同来源 token 之间的高阶作用。

直觉上它像：

“所有证据同时摆在桌上”，模型自己决定谁和谁应该更早发生互动。

9.2 HyFormer 会怎么做

HyFormer 更像先做两步：

第一步，基于候选 item、上下文、静态画像和序列粗摘要，先生成一组 query；

第二步，让这些 query 去读长序列，再带着读到的信息回来，与非序列特征做 boosting。

所以它的直觉不是“所有人一起开大会”，而是：

- 先派几位代表；
- 代表带着任务去读历史；
- 回来以后再和用户画像、item、上下文一起讨论；
- 下一层再派更成熟的代表继续读。

9.3 这两个直觉最本质的不同

- OneTrans：所有 token 一起聊；
- HyFormer：query 代表团分轮去问，再回来开会。

两者都在做统一建模，但统一发生的位置并不一样：

- OneTrans：统一在输入表示与主干结构上；
- HyFormer：统一在 query interface 与层级交互机制上。

10 如果把它们映射回你熟悉的 CTR / 排序框架

10.1 它们都在反对什么

它们都在反对这样一种旧思路：

先把长序列压成一个固定历史向量

-> 再和表格特征 late fusion

-> 再做一个二分类头

这套思路的问题不在于“完全没用”，而在于它把真正复杂的交互推迟到了最后。非序列上下文很难反过来影响“该保留哪段历史”，历史信息和字段信息也往往只在最后碰一次，结果就是前面两支各自学得都不差，但真正决定排序质量的细粒度联动并没有发生得足够早、足够深。再往工程上看，这种 late fusion 也不利于统一 scaling 和统一 serving 优化，因为你仍然要维护两套明显不同的中间表示与推理路径。

10.2 OneTrans 的映射方式

如果把 OneTrans 映射回大家熟悉的 CTR / 排序框架，它更像是在说：既然最终都要联合打分，那不如从一开始就把“序列编码”和“字段交互”都改写成统一 token 流上的同一种计算。候选 item、用户画像、上下文不再只是最终打分头前的附加输入，而是从底层开始就进入主干；DIN 里那种

“候选感知读历史”的动作，也被并入统一 attention 机制。换句话说，OneTrans 更接近“用一条 Transformer 主干重写 CTR 排序”。

10.3 HyFormer 的映射方式

HyFormer 的映射方式则更温和一些。它承认长序列侧仍然有自己相对独立的编码逻辑，但它不再满足于“先编码完、固定住、最后再拼”这种旧范式，而是通过 global query 在每一层里反复把“序列读取”和“异构交互”绑在一起。你可以把它理解成一种接口层面的统一：我承认长序列和异构表格本来就不完全同构，但我用 query 接口把它们组织成一个持续循环，而不是一次性晚融合。

10.4 对“统一建模”这个命题，它们分别给出的答案

如果要把两者各自的答案压缩成一句话，那么 OneTrans 的答案是：既然最终都要联合建模，那就尽可能统一 token 化、统一主干、统一 attention 信息流；HyFormer 的答案则是：统一不必等于全部同流，统一也可以是“接口统一 + 分工明确的层内循环”。但两者真正共享的共识其实比差异更重要，那就是不要再把序列和字段交互只做成一次性晚融合，而要让两边更早、更深、更反复地交换信息。

11 回到工业落地与 TAAC 这类赛题，该怎么理解

如果把这套框架放回大规模推荐竞赛或真实工业排序环境里，最关键的不是“论文名是不是新”，而是你能不能把模型思路和工程现实连起来理解。

11.1 第一，为什么这类任务天然适合统一建模

这类任务之所以天然适合统一建模，是因为它往往同时具备多字段离散特征、稠密特征、多域长序列、明确的主排序指标以及严格的推理时延约束。这样的任务会让你很快发现，只做 DLRM 往往不够，因为动态兴趣信息吃不饱；只做 SASRec、BST 这类序列方法也往往不够，因为多字段表格信号又吃不饱。题目真正想看的，不是哪一条老路线单独堆得更高，而是你能否把两类信息更自然地放到一个系统里。

11.2 第二，为什么单模型、单链路很重要

很多赛题会强调单模型、单条推理链路，而且时延不达标往往就直接无效。这其实是在逼你面对统一建模最真实的那一面：不是论文里说“统一”就够了，还得在 serving 时真的统一得动。只要规则里出现这些约束，你就不能再只从离线分数出发看问题，而必须同时考虑 schema、图结构、缓存策略和线上推理路径。

11.3 第三，不是 AUC 最大就一定能上线

工业线上不是只看 AUC，很多时候晚一点点就不能部署。所以真正重要的不是“裸 AUC”，而是“时延预算下的 AUC”。这也是为什么 OneTrans 会特别强调 KV cache、pyramid stack、统一算子

优化，而 HyFormer 会特别强调 sequence encoder 的容量-效率权衡、GPU pooling、异步通信。它们都不是在做“附加工程技巧”，而是在回答同一个问题：当模型真的要上线时，怎样把效果收益留住。

11.4 第四，统一建模不是让你一上来就堆超大模型

对大多数团队和参赛者，更现实的顺序通常是先证明非序列特征有效，再证明序列特征有效，然后再证明统一之后的额外增益确实来自“更深交互”，而不是“只是模型更大了”，最后再去做时延压缩和工程优化。这样做的好处是，每一步的收益来源都更容易解释，也更容易在后面做消融与系统决策。

12 训练策略应该怎么想

12.1 先明确你优化的到底是什么

如果任务目标是点式二分类排序，那最常见的还是：

$$p = \sigma(s), \quad \mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{n=1}^N \left[y_n \log p_n + (1 - y_n) \log(1 - p_n) \right].$$

但别忘了，训练 loss 和最终评估指标经常不是同一个东西。比如你可能训练时主要看 BCE，最终却更关心 AUC。这并不矛盾，因为 BCE 更适合做梯度优化，而 AUC 更贴近排序质量。真正需要避免的，不是“训练目标和评估指标不同”，而是你根本没想清楚二者各自负责什么。

12.2 数据切分一定要按时间

推荐里最危险的错误之一，就是验证集划分不严格。一个非常稳妥的原则是：

所有特征都必须能在预测发生当刻被真实拿到。

这句话一旦落到实验上，含义就非常具体了：验证集要按时间切分，序列构造只允许看到预测时刻之前的行为，未来统计量、未来用户画像、未来曝光信息都不能泄漏进来。很多看起来“模型涨了很多”的实验，最后复盘其实都是这个环节没守住。

12.3 序列构造要避免“偷看答案”

如果你预测的是某个时刻的点击或转化，那这条样本的历史序列应该只由该时刻之前的行为组成。很多实验的虚高，不是模型真强，而是序列里偷偷塞进了目标时刻之后的信息。

12.4 长序列不一定越长越好

长序列当然更完整，但代价也更大。序列一长，attention 开销会更高，padding 和 mask 会更复杂，噪声也可能更多，所以实践中往往要把“序列长度上限”当成一个真正的超参数来调，而不是默认越长越好。很多时候，一个被精心裁剪过的有效窗口，反而比“能塞多少塞多少”的长序列更

稳。

12.5 类别不平衡、多目标训练、时延约束都要谨慎

推荐任务里常见的点击或转化标签往往很不平衡，必要时可以考虑 `sample reweight` 或 `focal loss`，但对初始 `baseline` 来说，先把样本构造和验证切分做对，通常比一上来就折腾复杂 `loss` 更重要。工业系统里也经常会同时间优化 CTR、CVR、CTCVR、`dwell time` 等多个目标，不过如果主指标明确是单一 ROC-AUC 排序，初期最好不要把问题搞得过于复杂，而是先把单目标 `baseline` 做稳，再判断 `auxiliary task` 是否真的带来泛化收益，最后再权衡它是否增加训练和推理复杂度。

同样重要的是，训练时就要把时延纳入设计。序列长度上限、`hidden size`、`block` 层数、`attention` 复杂度、`embedding` 维度、稠密特征投影维度，这些都不是后面上线时才突然出现的工程参数，而是从建模阶段就应该被约束住的。一个非常现实的原则是：每次涨点都要问自己，这个涨点在推理预算下还能不能留下来。

13 评估策略应该怎么做

13.1 不要只盯着 leaderboard

公开榜不是最终榜，推荐比赛里尤其容易出现 `public leaderboard` 上涨了、但 `private leaderboard` 掉了的情况。原因通常无非就是过拟合验证集、过拟合公榜，或者线上分布本身发生了漂移。所以真正可靠的做法，不是天天盯榜波动，而是先建立自己的稳定离线验证，再把榜单当作附加信号来参考。

13.2 建议同时看这些指标

如果主目标是 AUC，离线至少也应该同时监控 `LogLoss`、正负样本分数分布、不同时间切片 AUC、不同人群分桶 AUC 以及推理时延。原因很简单，AUC 高不代表校准好，也不代表泛化稳定，更不代表线上跑得动。只有把这些指标放在一起看，你才知道模型到底是在“真正变好”，还是只是碰巧在某个局部验证集上变好。

13.3 做分桶评估

如果只看总 AUC，你经常不知道涨点从哪来，掉点又掉在哪。所以推荐按用户活跃度、序列长度、时间段、`item` 热度、新老用户以及行为域是否缺失来做分桶评估。这样一来，你就能看清模型是不是只对高活跃用户有效，是不是只对短序列有效，或者是不是一到冷门 `item` 就开始失效。很多“平均分不错”的模型，问题恰恰就藏在这些桶里。

13.4 做消融实验

统一建模最容易出现一个错觉：模型看起来很复杂，但其实真正有用的部分只有少数几个。所以消融实验一定要做，而且要做得足够直接。最基本的几种对照包括：去掉所有序列、只保留非序列特征；去掉所有非序列特征、只保留序列；去掉 `domain token` 或 `type token`；去掉 `position encoding`；

缩短序列长度；减少 backbone 层数；或者干脆把统一 backbone 改回双塔或双分支。只有这样，你才能真正回答涨点是不是来自统一建模本身，而不是因为模型更大了、参数更多了。

14 一个务实的实验起步路径

14.1 Baseline 1: 强表格基线

第一步最好先做一个偏特征交互的 baseline。具体来说，可以把离散字段做 embedding，把稠密特征做投影，再接多层 MLP 或 DLRM / DCN 风格交互，同时先把序列做成简单池化。这样做的优点是容易训练、工程简单，而且能很快搭出一个可提交系统；代价则是动态兴趣建模会比较弱。但对起步阶段来说，这种“先把表格主线打稳”的策略往往非常值。

14.2 Baseline 2: 加一个轻量序列模块

在 Baseline 1 之上，第二步可以加一个轻量序列模块，比如让候选 item 对历史做 target attention，或者加 1 到 2 层轻量 Transformer。这一步的目标不是追求“看起来更先进”，而是先确认序列信息在你的验证集上到底有多大价值。如果连这一步都拿不到稳定收益，那后面直接上统一建模通常只会让问题更难解释。

14.3 Baseline 3: 往统一建模推进

当你已经确认非序列特征有效、序列特征也有效，就可以开始往统一建模推进。比较自然的做法是把字段转成统一 token，为不同字段加入 field、type、domain、position embedding，再用统一 backbone 建模，最后只保留一个最终打分头。这样推进的好处是，你知道自己是在“整合两个已被证明有效的信息源”，而不是凭空把复杂度堆高。

14.4 为什么不建议新手一上来就上超大统一模型

因为一上来就上超大统一模型，你会同时遇到数据读取复杂、mask 复杂、序列截断复杂、训练不稳定、时延不可控、消融难解释等一整串问题。更好的顺序始终是先证明每类信息各自有价值，再证明统一之后额外增益来自“更深交互”，最后再做时延压缩和工程优化。这样每前进一步，实验结论都会更稳。

14.5 如果你准备开始做实验，最建议先做的 5 件事

如果你现在就准备开做，我最建议先做五件事，而且最好一开始就养成习惯。第一，先把样本结构彻底摸清楚，明确哪些字段是离散、哪些是稠密、哪些是序列；第二，先做一个不复杂但稳定的 baseline，别急着上大模型；第三，验证集按时间切分，严防泄漏；第四，每次实验都同时记录 AUC、LogLoss、序列长度、参数量和推理时延；第五，所有涨点都做消融，确认增益到底来自哪里。把这五件事做好，后面你读论文、改结构、看榜单时都会稳很多。

15 给初学者的一个最终心智模型

如果你现在只能带走一个框架，那就记住三句话就够了。第一，特征交互模型最擅长回答的是“当前这次曝光里的多字段组合关系是什么”；第二，序列模型最擅长回答的是“用户最近兴趣是怎么变化的，当前候选应该重点匹配哪段历史”；第三，统一建模想回答的是“能不能用一个单一、同构、可扩展、可部署的模型，把上面两个问题一起学掉”。OneTrans 和 HyFormer 给出的不是同一个结构答案，但它们给出的方向共识是一致的：不要只做晚融合，不要把序列建模和字段交互长期看成两件互不相干的事，真正重要的是让两边更早、更深、更持续地交换信息。

16 参考资料与建议阅读

16.1 经典路线

1. Wide & Deep Learning for Recommender Systems.
2. Deep Learning Recommendation Model for Personalization and Recommendation Systems.
3. Deep Interest Network for Click-Through Rate Prediction.
4. Deep Interest Evolution Network for Click-Through Rate Prediction.
5. Behavior Sequence Transformer for E-commerce Recommendation in Alibaba.
6. Self-Attentive Sequential Recommendation.
7. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer.

16.2 统一建模方向

1. Zhaoqi Zhang, Haolei Pei, Jun Guo, et al. *OneTrans: Unified Feature Interaction and Sequence Modeling with One Transformer in Industrial Recommender*. arXiv:2510.26104, 2025.
2. Yunwen Huang, Shiyong Hong, Xijun Xiao, et al. *HyFormer: Revisiting the Roles of Sequence Modeling and Feature Interaction in CTR Prediction*. arXiv:2601.12681, 2026.

16.3 建议怎么读

如果你现在刚入门，更推荐下面这个顺序：

1. 先读 LR / FM / Wide & Deep / DLRM，理解“特征交互路线”；
2. 再读 DIN / DIEN / BST / SASRec / BERT4Rec，理解“序列路线”；
3. 最后再读 OneTrans / HyFormer，这样你看到的就不只是新名词，而会知道它们到底在统一什么。

16.4 最后一段话

“两条路线单独演进”不是一句玄学黑话。它真正的意思只是：

- 过去很多系统里，表格特征和行为序列是分开建模的；

- 现在大家越来越想把它们放进同一个模型里联合学习。

OneTrans 和 HyFormer 只是用两种不同方式，把这个趋势讲得更彻底了一点。